

3. Übung

Ausgabetermin: 21.05.2026

Abgabetermin: 11.06.2026

Gesamtpunkte:

250 P

In dieser Übung sollen Sie ein Spiel in C++ mit Qt implementieren. Es handelt sich um einen klassischen Minesweeper. Der Spieler soll in einem $(n \times m)$ -Spielfeld alle verdeckten Felder aufdecken oder markieren, ohne eine Mine zur Explosion zu bringen. Der Spieler hat gewonnen, wenn er alle Minen markiert und alle übrigen Felder aufgedeckt hat.

Aufgabe 1

(80 P)

Benötigt wird ein Hauptfenster, in der das Spielfeld angezeigt wird. Es sollte über eine Statuszeile verfügen, die anzeigt, wieviele Minen enthalten sind, wieviele Marken bereits gesetzt wurden, und wieviel Zeit seit Spielbeginn vergangen ist. Es sollte über drei Schalter „Neu“, „Pause“ und „Beenden“ zum Starten, Pausieren und Beenden eines Spiels verfügen. Schließlich sollte das Fenster noch über ein Menü verfügen, über das ein Einstellungsfenster, ein Statistikfenster und ein Hilfefenster geöffnet werden können.

Die Größe des Hauptfensters soll eine Mindestgröße nicht unterschreiten, aber sich der Größe des $(n \times m)$ -Spielfeldes anpassen, die wiederum von den Parametern n und m abhängt.

In dem Spielfeld sollten die einzelnen Felder klar von ihren Nachbarfeldern abgegrenzt sein. Die einzelnen Felder können verschiedene Zustände annehmen:

- verdeckt: das Feld ist dunkel gehalten; es existieren folgende Unterzustände, die für den Spieler nicht unterscheidbar sind:
 - das Feld enthält eine Mine
 - das Feld enthält keine Mine
- markiert: das Feld ist hell gehalten und enthält eine Marke (bspw. eine kleine Flagge oder einfach das %-Zeichen)
- aufgedeckt: das Feld ist hell gehalten; es existieren folgende Unterzustände:
 - unvermint: es ist keine Mine enthalten; es existieren zwei Unterzustände:
 - * das Feld grenzt an keine verminten Felder an: das Feld ist unbeschriftet
 - * das Feld grenzt an p verminten Felder an: das Feld ist mit dem Wert p beschriftet
 - vermint: das Feld enthält eine Mine (bspw. eine kleine Bombe oder einfach den schwarzen Buchstaben X)
 - explodiert: das Feld enthält eine explodierte Mine (bspw. eine kleine Explosion oder einfach den roten Buchstaben X)

Mit jedem einzelnen Feld sind folgende Interaktionen möglich, die die angegebenen Konsequenzen nach sich führen:

- linker Mausklick: das Feld wird aufgedeckt
dabei geht das Feld in einen der o. a. Unterzustände über; ist das Feld vermint, explodiert die Mine und alle Felder werden aufgedeckt; ist das Feld unvermint und grenzt an kein vermintes Feld, so werden die Nachbarnfelder ebenfalls aufgedeckt (dies geschieht rekursiv)
- rechter Mausklick: das Feld wird markiert
in der Statuszeile wird die Anzahl der Marken aktualisiert; sind mehr Marken gesetzt, als Minen vorhanden sind, wird die Anzahl der Marken in Rot angezeigt, sonst in Blau
- mittlerer Mausklick: die Markierung eines markierten Feldes wird wieder entfernt
in der Statuszeile wird die Anzahl der Marken aktualisiert; sind mehr Marken gesetzt, als Minen vorhanden sind, wird die Anzahl der Marken in Rot angezeigt, sonst in Blau

Bei Betätigen der Schalter werden folgende Aktionen ausgelöst:

- **Neu:** ein neues, vollständig verdecktes $(n \times m)$ -Spielfeld wird erzeugt und angezeigt. In dem Spielfeld sind k Minen zufällig verteilt, die aber nicht angezeigt werden. Die Zeit in der Statuszeile wird auf Null gesetzt.
- **Pause:** Ein laufendes Spiel wird pausiert: die Zeit wird in der Statuszeile angehalten und das Spielfeld nicht mehr angezeigt. Ein pausiertes Spiel wird weiterlaufen gelassen: die Zeit läuft wieder und das Spielfeld wird wieder angezeigt.
- **Beenden:** Alle Felder werden aufgedeckt und das Spiel als verloren gezählt.

Das Spiel ist beendet wenn eine der folgenden Situationen eintritt:

- der Beenden-Schalter betätigt wurde; das Spiel zählt als verloren
- das letzte verdeckte Feld aufgedeckt oder markiert wurde; das Spiel zählt als gewonnen, wenn genau die verminten Felder markiert wurden, sonst zählt es als verloren
- ein vermintes Feld wurde aufgedeckt; das Spiel zählt als verloren

Bei Spielende werden alle falsch markierten Felder in Rot angezeigt, alle korrekt markierten Felder in Blau.

Entwerfen Sie das entsprechenden Fenster mittels des Qt-Designers. Wählen Sie die einzelnen Komponenten und ihr Layout unter softwareergonomischen Gesichtspunkten.

Aufgabe 2 (50 P)

Es werden noch drei weitere Fenster benötigt:

Das Einstellungsfenster soll ermöglichen die Werte für n , m und k zu konfigurieren und über einen Bestätigen- und einen Abbruch-Schalter verfügen.

Das Hilfefenster sollte darüber informieren, welche Aktionen durch die Maustasten ausgeführt werden.

Das Statistikfenster sollte für jede Einstellung von n , m und k anzeigen, wieviele Spiele mit dieser Einstellung gespielt wurden, wieviele gewonnen bzw. verloren wurde, und welches die kürzeste Zeit war, in der gewonnen wurde. Wählen Sie eine geeignete grafisch-textuelle Darstellung.

Entwerfen Sie die entsprechenden Fenster mittels des Qt-Designers. Wählen Sie die einzelnen Komponenten und ihr Layout unter softwareergonomischen Gesichtspunkten.

Aufgabe 3 (80 P)

Implementieren Sie geeignete objektorientierte Datenstrukturen, um das Spielfeld mit seinen einzelnen Feldern und deren Zuständen darstellen und die Zustandsübergänge umsetzen zu können, sowie um die Spielstatistiken führen zu können. Beachten Sie die Anforderungen aus den Aufgaben 1 und 2. Kommentieren Sie Ihre Implementation ausführlich.

Aufgabe 4**(30 P)**

Implementieren Sie ein Qt-Programm, das Ihre Oberflächen aus Aufgaben 1 und 2 nutzt, die dort beschriebenen Funktionalitäten umsetzt, und Sie mit Ihren Datenstrukturen aus Aufgabe 3 verbindet. Kommentieren Sie Ihr Programm ausführlich. Stellen Sie sicher, dass Ihr Programm ordnungsgemäß auf den Poolrechnern der Hochschule unter Linux unter Verwendung der Qt-Pipeline mit `qmake` und `make` compiliert und funktioniert.

Aufgabe 5**(10 P)**

Beschreiben Sie auf ein bis zwei Seiten in einem PDF-Dokument die Funktionsweise Ihres Programms.

Hinweise

Benennen Sie Ihre Klassen, Funktionen und Variablen sinnvoll mit sprechenden Namen. Kommentieren Sie Ihre Implementation ausführlich. Jede Klasse muss einen Kommentar enthalten, der den Autor der Klasse nennt und den Zweck der Klasse kurz beschreibt. Jede Funktion muss einen Kommentar enthalten, der den Autor der Funktion nennt und die Eingabe- und Rückgabeparameter sowie eventuelle Seiteneffekte beschreibt. Beachten Sie, dass die Kommentierung stets in die Bewertung eingeht.

Erzeugen Sie eine zip-Datei, die Ihre vollständigen zu diesem Projekt gehörigen Dateien und notwendige Dokumentation enthält, und laden Sie sie auf OLAT zu der zugehörigen Übung unseres Kurses hoch.